Witness Separation for Hard Attention over Boolean Cubes: A Machine-Checked Expressivity Theory in Lean 4

Ben Cassie
Independent Researcher
bencassie@outlook.com
ORCID: 0009-0004-1899-7627

Published on Zenodo. DOI: 10.5281/zenodo.21188381.

Abstract

We present `attention-lean`, a Lean 4 / Mathlib library developing a machine-checked expressivity theory for a specific model of single-layer hard attention over finite Boolean cubes. The library proves exact head-count results for concrete Boolean functions: parity on n bits requires n heads under any thresholded affine readout (`parityN_requires_N_heads`), and strict majority on five bits requires exactly four heads (`maj5_head_number_exact`). The engine behind these bounds is a *witness separation* method: every head output is a `Fixable` witness — a function whose value on every subcube is forced by a single literal — and any family of witnesses that collides on inputs separated by the target rules out every aggregator at once. We prove the class of single-head output functions is *exactly* the class of decision lists (`head_output_iff_fixable`, `fixable_iff_dl`), so head-count questions reduce to a combinatorial witness number $k(T)$. The flagship combinatorial result is the first gap between witness number and certificate complexity: $k(\mathrm{maj}_5) = 4$ while three agreeing votes certify majority on five bits (`maj5_witness_number_exact`), so witness complexity is not certificate complexity, and the certificate rung of the lower-bound ladder is a strict lower bound only. All headline theorems are machine-checked with axiom footprint exactly `{propext, Classical.choice, Quot.sound}`, no `sorry`, and no `native_decide` on the clean tier; a compile-time gate (`lake exe axiom_check`) pins 103 declarations' footprints so that any drift fails the build. We describe the model, the witness method, the proof architecture of the majority gap — an oracle-guided but kernel-re-verified case analysis over classified threshold catalogs — and the development discipline used to avoid hallucinated API names during AI-assisted formalisation.

1. Introduction

Attention-based architectures dominate applied machine learning, and a growing theory literature asks what individual attention layers can and cannot compute. This paper contributes a small, fully machine-checked corner of that programme: exact head-count bounds for a *specific, deliberately narrow* model — single-layer, unique-argmax ("hard") attention over fixed-length Boolean inputs, with per-head Boolean outputs combined by a thresholded affine readout.

The scope caveat comes first, because the temptation to over-read such results is real. Nothing in this paper is a statement about transformers as deployed: we do not model soft attention, multiple layers, positional encodings beyond the token embedding, numerical precision, or learned behaviour. When we write "majority on five bits requires four heads," the claim is a theorem about the model of Section 2 and nothing larger. What the narrowness buys is completeness: within the model, the bounds are exact, the expressivity class of a single head is characterised exactly, and every claim is checked by the Lean kernel down to a fixed three-axiom footprint.

The library's contributions, in the order the paper presents them:

1. **A witness separation engine** (Section 3). Head outputs are `Fixable` witnesses; targets that separate inputs on which all witnesses collide are not computable by *any* aggregator, thresholded-affine or otherwise. Computability by k witnesses is characterised as refinement of the target by the joint witness map.
2. **Parity bounds** (Section 4). Parity on n bits requires n heads; 2^{n-1} heads suffice; at $n = 3$ the exact head count is four, proved without enumeration on the clean tier.

3. **The majority gap** (Section 5). The witness number of strict majority on five bits is exactly four, strictly above its certificate complexity of three — the first function in the library's ladder where the certificate lower bound is not tight. The proof is a kernel-checked case analysis over a classified catalog of threshold-refining witness pairs.
4. **The bridge theorem** (Section 6). Single-head expressivity is exactly the decision-list class, in both directions; consequently k-head computability under arbitrary aggregators coincides with k-witness computability, and the majority gap transfers to heads: $k_{\text{heads}}(\text{maj}_5) = 4$.
5. **A verified artifact** (Sections 7–8) with a compile-time axiom gate and an explicit clean/dirty tier separation.

2. The model

The model lives in `AttentionLean/Defs.lean`. A head over n Boolean positions with internal dimension d is a structure of real score matrices, a query vector, a token embedding, a value-projection vector, and an affine readout:

```
structure HardAttentionHead (n d : ℕ) where
  W_Q : Matrix (Fin d) (Fin d) ℝ
  W_K : Matrix (Fin d) (Fin d) ℝ
  query : Fin d → ℝ
  tok : Fin n → Bool → (Fin d → ℝ)
  W_V : Fin d → ℝ
  readout_w : ℝ
  readout_b : ℝ
```

On input $x : \text{Fin } n \to \text{Bool}$, the attention score at position i is $\langle W_Q q,\ W_K \,\text{tok}(i, x_i)\rangle$. The score is *position-local*: it depends only on the pair (i, x_i), a fact recorded once as `attentionScore_eq_scoreVal` and used everywhere. The head attends to the argmax position, with ties broken to the smallest index (`argmaxScore`); it reads the value $\langle W_V, \text{tok}\rangle$ at the winner and thresholds an affine function of it, giving a Boolean output `headOutput`. A family of k heads computes a target T through a thresholded affine readout when

$$\left[\textstyle\sum_i w_i \cdot [\text{headOutput}(h_i, x)] + b > 0\right] = T(x) \quad \text{for all } x.$$

The combinatorial abstraction that drives all lower bounds is the following property of Boolean functions (`Fixable`, in `ParityN.lean`). A *subcube* is a partial assignment $\rho : \text{Fin } n \to \text{Option Bool}$; membership `memCube ρ x` says x matches every pinned coordinate. A function f is *fixable* when on every subcube some non-excluded literal forces it:

```
def Fixable (f : (Fin n → Bool) → Bool) : Prop :=
  ∀ ρ, ∃ i b, ρ i ≠ some (!b) ∧
    ∃ c, ∀ x, memCube ρ x → x i = b → f x = c
```

The connection to the model is `headOutput_fixable`: **every head output is fixable**. On any subcube, among the literals not excluded by the pins, the one with maximal score (ties to the smallest index) pins the argmax; fixing that literal makes the head constant on the restricted subcube. This is where the model's structure — position-local scores plus deterministic argmax — enters, and it is the only place it needs to enter: everything downstream is combinatorics of fixable functions.

We write $k(T)$ for the *witness number* of a target T: the least k such that $T = g(w_1, \ldots, w_k)$ for fixable witnesses w_i and an arbitrary aggregator $g : \text{Bool}^k \to \text{Bool}$. Because the aggregator is unrestricted, lower bounds on $k(T)$ are stronger than lower bounds against any particular readout; Section 6 shows the reduction is lossless in the other direction as well.

3. Witness separation: the spine

The reusable core of every lower bound is one observation (`witness_separation_fails`, in `WitnessSeparation.lean`): if two inputs x, y receive the same value from *every* witness but different values from the target, then no aggregator computes the target — the aggregator sees identical arguments and must answer identically. The statement is fully generic (arbitrary state, value and output types) and its axiom footprint is `Quot.sound` alone.

The companion characterisation (`witness_computable_iff_refines`, in `WitnessTheory.lean`) says the collision obstruction is the *only* obstruction: a target is computable from witnesses $w_1, \ldots, w_k$ by some aggregator if and only if the joint witness map $x \mapsto (w_1(x), \ldots, w_k(x))$ *refines* T, i.e. equal joint labels force equal target values. Non-computability is thus always witnessed by a concrete collision pair (`witness_separation_fails_of_char`). All the ladder's lower bounds are instances of one recipe: use fixability to build a subcube on which every witness is constant while the target is not, then exhibit the collision.

The instrument for building such subcubes is a pinning induction (`fixable_pin_list`): processing k fixable witnesses costs at most k pinned coordinates — each witness's forcing literal is pinned in turn, and literals already pinned cost nothing. A quantitative companion, the counting bound `witness_counting_bound`, bounds the image cardinality of a k-witness map, but plays no role in the head bounds; we state it for completeness of the theory and to record honestly that counting alone does not recover any of the bounds below.

4. Parity

Parity on n bits (`parityN`) is *everywhere sensitive*: flipping any coordinate anywhere flips the value. The general lower bound (`fixable_witnesses_lower_bound`) says an everywhere-sensitive target needs at least n fixable witnesses: with $k < n$ witnesses, the pinning induction leaves a free coordinate, whose flip toggles parity inside a subcube on which every witness is constant — a collision. Composed with `headOutput_fixable` and the readout-as-aggregator observation, this yields the head bound `parityN_requires_N_heads`:

Theorem (parity lower bound). For every n and $k < n$, no k hard-attention heads with a thresholded affine readout compute parity on n bits.

`parityN_requires_N_heads_of_witness_theory` records the same statement factored through the witness theory, so that the head bound is literally an instance of the fixable-witness bound.

The bound is not a pigeonhole fact about arbitrary Boolean features — $n-1$ arbitrary witnesses *can* compute parity through an unrestricted aggregator is false, but $n-1$ arbitrary Boolean functions can (unary counting); the theorem depends essentially on fixability. On the upper side, `parityN_achievable_with_exp_heads` constructs 2^{n-1} indicator heads (one per odd point; `indicatorHead`) computing parity through a linear-threshold readout, and `parity2_achievable_with_two_heads` closes $n = 2$ exactly. At $n = 3$ the library proves the exact head count is four: `parity3_not_achievable_with_three_heads` is a structural, enumeration-free proof on the clean tier, paired with an explicit four-head construction in `parity3_head_complexity_four`. So the witness number of parity is exactly n (`parityN_witness_number_exact` — dictator witnesses with a parity aggregator are optimal), while the *head* count under affine readouts exceeds it already at $n = 3$: the witness-to-head transfer of upper bounds is aggregator-sensitive, a point Section 6 returns to.

5. Majority and the first gap

Majority is not everywhere sensitive, so the parity engine does not apply — deliberately so; the interest of majority is that it probes which measure actually governs the witness number. The certificate-style bound (`fixable_witnesses_lower_bound_of_nonconstant` instantiated as `maj_needs_half_fixable_witnesses`) shows that if a target is non-constant on every subcube with at most k pins, it needs more than k fixable witnesses; majority on n bits is non-constant on every subcube with $2k < n$ pins (`maj_nonconstant_on_small_subcubes`), giving $k(\mathrm{maj}_n) > n/2$. A restriction-embedding rung (`restriction_embedding_lower_bound`, with crux `fixable_restrict`: fixability survives restriction) extends the ladder past everywhere-sensitivity: inner product modulo two on $2m$ bits needs m witnesses (`ip2_needs_m_fixable_witnesses`) although it is not everywhere sensitive, and the bound is exact (`ip2_witness_number_exact`).

For small majorities the certificate bound is tight: $k(\mathrm{maj}_3) = 2$ (`maj3_witness_number_exact`), matching the size of a smallest certificate (two agreeing votes). The first interesting case is maj_5, where three agreeing votes certify the value, so the certificate measure says only $k \geq 3$.

Theorem (the first gap). `maj5_witness_number_exact`: four fixable witnesses and an aggregator compute strict majority on five bits (`maj5_computable_by_four_fixable`, with the explicit witnesses $w_1 = x_0 \wedge (x_1 \vee x_3 \vee (x_2 \wedge x_4))$ and its three mates, and aggregator $(v_0 \wedge v_1) \vee v_2 \vee v_3$; the equality is a kernel `decide` over the 32 cube points, `maj5_eq_four_witness_combination`), and no three fixable witnesses compute it under any aggregator (`maj5_no_three_fixable_witnesses`). Hence $k(\mathrm{maj}_5) = 4 > 3$: **witness number is not certificate complexity**, and the certificate rung is a strict lower bound only.

The lower half is the deepest proof in the library, and its architecture is worth recording. A structural reduction (`maj5_reduction`) first shows any three fixable witnesses computing maj_5 carry constancy half-cubes whose signed directions are pairwise distinct, with unanimous signs whenever the directions are pairwise distinct — killing outright the configurations where two witnesses share a signed direction (`maj5_shared_face_kill`) or three distinct directions carry mixed signs (`maj5_mixed_signs_kill`). Two configurations survive. Both are dispatched through a classification theorem for the faces they force: any fixable pair of functions refining the four-bit threshold T_2^4 (at least two of four) is one of exactly 24 catalog pairs (`T2_refining_pair_classified`), and dually for T_3^4 (`T3_refining_pair_classified`); the catalog cardinalities are themselves kernel facts (`catalog_card`, `catalog3_card`). With the catalogs available, the surviving witnesses become piecewise-catalog functions of finitely many parameters, and kernel `decide`s over those parameter spaces finish both cases: in the shared-direction configuration the doubly-classified third witness is unfixable for *all* 2,880 parameter choices, and in the distinct-direction configuration each witness is either unfixable or lands on an explicit sixteen-entry exceptional list per sign — and the three exceptional lists are mutually incompatible on the face subsets a genuine triple must share.

Methodologically, the case analysis was *designed* against an exhaustive search, `maj5_witness_search.py` under `scripts/`, whose positive controls reproduce $|\mathrm{Fixable}(3)| = 96$ — also a kernel fact, `card_fixable3` — and the known maj_3 constructions) and then *re-verified in the kernel*: every count the search produced is re-established by a `decide` inside the Lean proof, so the search is corroboration, not evidence. One count remains script-only and we flag it as such: the total $|\mathrm{Fixable}(4)| = 1050$ is beyond kernel-`decide` range (it quantifies a 2^{16}-function space against an 81-subcube test) and is *not* used by any theorem.

Underlying the catalog classification is a normal form of independent interest (`fixable_iff_dl`, in `FixableNormalForm.lean`): a function is fixable if and only if it is computed by a decision list — a chain of single-literal tests with constant outputs and a constant default. The forward direction extracts the forcing literal at the empty restriction and recurses through `fixable_update_restrict` (pinning one coordinate of a fixable function is fixable); the reverse pins the head literal or transfers the tail's.

6. The bridge theorem

The final theory drop closes the loop between the combinatorial witness theory and the attention model, in both directions (`DecisionListHeads.lean`).

Theorem (bridge). `head_output_iff_fixable`: a Boolean function on n bits is the output function of some hard-attention head if and only if it is fixable — equivalently, by `fixable_iff_dl`, if and only if it is a decision list. Moreover internal dimension two always suffices for the forward direction.

The construction (`dl_realizable_by_head`, via `priorityDL_realizable`) realises any decision list as a *table head*: identity score matrices, coordinate-projection query and value vectors, and a token embedding carrying a (score, read) pair per literal (`tableHeadN`, generalising the concrete heads of `WitnessMaj5HeadsExact.lean`). The list's literals receive strictly decreasing positive scores in list order; unlisted and complementary literals fall through to score zero and read the default. On any input the argmax lands on the first live listed literal, or — when the whole list misses — on the score-zero tie broken to position zero, whose read is the default. The fall-through tables make well-formedness side conditions unnecessary: duplicated or dead literals resolve identically in the evaluator and in the argmax, both taking the first occurrence.

Two consequences. First, for *arbitrary* aggregators the head and witness pictures coincide exactly (`heads_computability_iff_fixable_witnesses`): a target is computable by k heads and an aggregator iff it is computable by k fixable witnesses and an aggregator. Every witness upper bound transfers to heads at dimension two, and every head lower bound may be proved purely in witness space. Second — the caveat, stated here exactly as in the theorem documentation — with a thresholded *affine* readout on the head side, upper bounds transfer only when the witness-side aggregator is itself threshold-affine: arbitrary aggregators are strictly more general than affine readouts, as the parity case already shows (n dictator witnesses compute parity through the XOR aggregator, which no affine readout simulates at that head count).

For majority the aggregator $(v_0 \wedge v_1) \vee v_2 \vee v_3$ *is* threshold-affine ($v_0 + v_1 + 2v_2 + 2v_3 > 3/2$), so exactness transfers all the way to the model:

Theorem (head-level exactness). `maj5_head_number_exact`: four hard-attention heads with a thresholded affine readout compute strict majority on five bits (`maj5_computable_by_four_heads`, the four decision-list heads `headW1_computes` … `headW4_computes`), and three cannot (`maj5_requires_four_heads`, an instantiation of `maj5_no_three_fixable_witnesses` through `headOutput_fixable`). Witnessed non-vacuity accompanies the negative half: a concrete family of three indicator heads fails against every aggregator (`maj5_three_indicator_heads_fail`), and even the shipped construction's own first three witnesses cannot be completed by any aggregator (`maj5_first_three_witnesses_fail`).

So within the model, $k_{\text{heads}}(\text{maj}_5) = k(\text{maj}_5) = 4$, strictly above the certificate complexity 3. As a piece of context we record without kernel backing: exhaustive (unformalised) computation of classical measures on the ladder's functions finds decision-tree rank, certificate complexity, sensitivity and block sensitivity all equal to 3 at maj_5; the witness number's separation from all of them at a single five-bit function, with the sandwich rank $= 3 < k = 4 <$ junta $= 5$, is what motivates the measure-theoretic reading of the result. Those comparison values are exhaustively computed but not formalised.

7. The verified artifact

The library is 40 Lean modules, roughly 8,300 lines, developed against the pinned toolchain `leanprover/lean4:v4.28.0` with the matching Mathlib release.

The clean tier and the gate. Every headline theorem carries the axiom footprint `{propext, Classical.choice, Quot.sound}` or a subset (several tools are `propext`-only or axiom-free, e.g. the reindexing transport `fixable_reindex`). There is no `sorry` anywhere in the library, and no `native_decide` on the clean tier: all finite checks are kernel `decide`, including the heavy catalog and case-analysis decides of Sections 5–6. The guarantee is enforced at compile time rather than by inspection: `AttentionLean/Axioms.lean` pins 103 declarations with `#guard_msgs in #print axioms` assertions, and the `axiom_check` executable is in the Lake default targets, so a bare `lake build` fails if any pinned footprint drifts — including the appearance of `sorryAx` or `Lean.ofReduceBool` anywhere in a pinned theorem's dependency cone.

The dirty tier, isolated by design. Two early fixed-width results — `parity3_requires_three_heads` and `parity4_requires_four_heads` — were proved by `native_decide` enumeration over achievable head behaviours and therefore carry `Lean.ofReduceBool` and `Lean.trustCompiler`. They are deliberately quarantined: pinned with their full expected (dirty) footprints in the companion `AttentionLean/AxiomsDirty.lean`, which is built (and hence gated against `sorry` creep) by a bare `lake build`, but excluded from the clean gate and from every clean-tier proof path. The clean-tier parity-3 result of Section 4 supersedes the enumerated one at $n = 3$ without touching it.

Evidence artifacts. The computational searches that guided the majority proof are committed under `scripts/` with their expected outputs documented, so the oracle-then-kernel workflow is reproducible: `maj5_witness_search.py` (the original exhaustive search) and `maj5_case_kill_oracle.py` (the case-kill design oracle whose every count is re-verified by kernel decides).

8. Methodology

The library was developed AI-assisted, and two disciplines from that workflow seem worth recording, in the spirit of the `kuramoto-lean` report.

Check before cite. Every API name — Mathlib lemma, project declaration — is verified by `#check` or `grep` against the sources before any proof or prose depends on it, and every new theorem's axiom footprint is probed with `#print axioms` *before* its expected footprint is pinned in the gate. Pinned footprints are never guessed; the pin records an observed fact. The same rule governed this paper: every theorem name cited above was grepped against the frozen sources at commit time.

Freeze before prove. Each module's headline statements were frozen in its docstring before proof work began, with explicit honest-stop clauses: when a target was refuted (an early conjecture that three heads compute three-bit parity died against the achievability search) or out of reach in a work window, the frozen text records the negative or the open boundary rather than a weakened substitute. Oracle-guided finite case analyses were validated against positive controls (known constructions, known cardinalities) before any Lean was written, then re-verified in the kernel so that no theorem depends on a script.

9. Related work

On the theory side, the limitations of self-attention for formal languages and counting go back to Hahn's study of soft attention (Hahn 2020, TACL) and the circuit-complexity upper bounds for saturated and hard attention (Merrill, Sabharwal and collaborators, 2021–2023); unique-hard-attention transformers have been characterised in formal-language terms (Angluin, Chiang and colleagues, 2022–2023). Those results concern uniform language classes for full architectures; our contribution is orthogonal in style — exact, finite, per-function head counts in a fixed single-layer model, with machine-checked proofs. The decision-list normal form connects the model to classical learning-theoretic classes (Rivest 1987) and to the rank measure of Ehrenfeucht and Haussler (1989); the witness-versus-certificate gap of Section 5 is, to our knowledge, not an instance of a known separation among the standard measures (the computed comparison of Section 6 documents the disagreement with rank, certificate complexity and sensitivity at maj_5), though we make no priority claim beyond the formalisation itself. On the formalisation side, machine-checked expressivity results for attention models are scarce; the development discipline follows the finite-dimensional, axiom-gated style of the author's `kuramoto-lean`.

10. Limitations and future work

The model is a single hard-attention layer with position-local scores, deterministic smallest-index tie-breaking, Boolean per-head outputs, and a thresholded affine readout; all bounds are exact *for this model*. Soft attention, multiple layers, richer value spaces and learned parameters are all out of scope, and we expect the numerology (though not necessarily the witness method) to change under any of these relaxations. The tie-breaking convention is load-bearing for the fixability of head outputs; alternative conventions would need the argument revisited. On the combinatorial side, the ladder's next open questions are concrete: the witness number of maj_7 is known only to lie in $[4, 7]$, and whether the gap between witness number and decision-tree rank grows with n is open. The general classification machinery (the decision-list normal form and the threshold catalogs) was built to make such questions finitely checkable; whether it scales past five bits without new ideas is genuinely unclear. Finally, the affine-readout transfer caveat of Section 6 delimits what the bridge theorem gives: exact head counts under affine readouts require, on the positive side, threshold-affine aggregators, and we have not investigated the head counts that arbitrary non-affine readouts would give for targets beyond those treated here.

11. Conclusion

Within a deliberately narrow hard-attention model, the library gives a complete, machine-checked account of single-head expressivity (exactly the decision lists), a reusable witness-separation engine for head-count lower bounds, exact head numbers for parity and five-bit majority, and a genuine separation: the witness number of maj_5 exceeds its certificate complexity. Every headline claim is kernel-checked on a three-axiom

footprint behind a compile-time gate. The theory is frozen as of this release; we hope the witness method and the artifact discipline are the reusable parts.

Artifact availability

Source: `github.com/velvetmonkey/attention-lean` (Zenodo deposit DOI: 10.5281/zenodo.21188381). Build: `lake build` with `elan`-pinned `leanprover/lean4:v4.28.0`; the axiom gate runs as part of the default targets and via `lake exe axiom_check`. This paper: `bash build-pdf.sh` (pandoc + tectonic + JuliaMono).